The `TestServer` that provides the in-memory server can be used for testing your real app, but in principle, a lot more configuration is required. Your real app likely loads configuration files or static files; it may use Razor Pages and views, as well as using `WebApplicationBuilder` instead of the generic host. Fortunately, the Microsoft.AspNetCore.Mvc.Testing NuGet package and `WebApplicationFactory` largely solve these configuration problems for you.

NOTE Don't be put off by the Mvc in the package name; you can use this package for testing ASP.NET Core apps that don't use any MVC or Razor Pages services or components.

You can use the `WebApplicationFactory` class (provided by the Microsoft.AspNetCore.Mvc.Testing NuGet package) to run an in-memory version of your real application. It uses the `TestServer` behind the scenes, but it uses your app's real configuration, DI service registration, and middleware pipeline. The following listing shows an example that tests that when your application receives a `"/ping"` request, it responds with `"pong"`.

Listing 36.8 Creating an integration test with `WebApplicationFactory`

```
public class IntegrationTests:                                    ❶
    IClassFixture<WebApplicationFactory<Program>>                 ❶
{
    private readonly WebApplicationFactory<Program> _fixture;     ❷
    public IntegrationTests(                                      ❷
        WebApplicationFactory<Startup> fixture)                   ❷
    {                                                             ❷
        _fixture = fixture;                                       ❷
    }                                                             ❷

    [Fact]
```